

Resolucion del Practico 6

Testing de Software 2024

Índice

Introduccion	2
1. Ejercicio 1: <code>Palindrome.capicua</code>	2
1.1. Codigo bajo prueba	2
1.2. Suite JUnit propuesta	3
1.3. Reporte de PIT	4
1.4. Respuestas	4
2. Ejercicio 2: <code>TriTyp.triang</code>	5
2.1. Codigo bajo prueba	5
2.2. Configuracion necesaria de PIT	5
2.3. Suite construida	5
2.4. Reporte de PIT	6
2.5. Respuestas	6
3. Ejercicio 3: Fuzzing	6
3.1. Estructura del paquete <code>fuzzing</code>	6
3.2. (a) <code>Mutator</code>	7
3.3. (b) <code>MutationFuzzer</code>	7
3.4. (c) <code>RandomFuzzer</code>	8
3.5. (d) <code>LinuxCommandTest</code>	9
3.6. Ejecucion	9
Conclusiones	10

Introduccion

Este documento contiene la resolucion del *Practico 6* de Testing de Software, centrado en dos tecnicas modernas de generacion y evaluacion de tests:

- **Testing de mutacion** con la herramienta Pitest. En los Ejercicios 1 y 2 se generan mutantes sobre el codigo bajo prueba (`Palindrome.capicua` y `TriTyp.triang`), se construye una suite JUnit que pretende matarlos a todos y se analiza el reporte de PIT (score, sobrevivientes, equivalentes).
- **Fuzzing** en el Ejercicio 3. Se completan los componentes `Mutator`, `MutationFuzzer` y `RandomFuzzer` y se cierra un test parametrizado que ejecuta el comando `bc` de Linux usando entradas generadas por mutacion.

Convenciones.

- **KILLED**: mutante muerto por al menos un test.
- **SURVIVED**: mutante sobreviviente (no es matado por ningun test).
- *Mutation Score* bruto: $\frac{\#KILLED}{\#GENERATED}$.
- *Mutante equivalente*: mutante cuyo comportamiento observable es identico al del programa original; no puede matarse y por eso no debe contar contra el score.

Como se ejecuta PIT. Desde `tp6/assignment-6-rodeghiero`:

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -Dmaven.repo.local
=.m2 \
-Djacoco.skip=true test org.pitest:pitest-maven:
mutationCoverage
```

La flag `-Djacoco.skip=true` desactiva el agente de JaCoCo (incompatible con la JVM 17 con la version del plugin del template); ademas, PIT instrumenta el bytecode por su cuenta, asi que no se pierde nada al saltarlo.

1. Ejercicio 1: `Palindrome.capicua`

1.1. Codigo bajo prueba

```
public static boolean capicua(char[] list) {
    int index = 0;
    int l = list.length;
    boolean res = true;
    while (index < (l - 1) && res) {
```

```

        if (list[index] != list[(l - index) - 1]) {
            res = false;
        }
        index++;
    }
    return res;
}

```

El metodo recorre el arreglo desde el extremo izquierdo y compara cada caracter con su *espejo* en el extremo derecho hasta cruzar el centro o detectar un par desigual.

1.2. Suite JUnit propuesta

```

@Test public void testCapicua() {
    char[] a = {'n','e','u','q','u','e','n'};
    assertTrue(Palindrome.capicua(a));
}
@Test public void testNoCapicuaPar() {
    assertFalse(Palindrome.capicua(new char[]{'a','b'}));
}
@Test public void testNoCapicuaConExtremosIguales() {
    assertFalse(Palindrome.capicua(new char[]{'a','b','c','a'}));
}
@Test public void testArregloVacioEsCapicua() {
    assertTrue(Palindrome.capicua(new char[]{}));
}
@Test public void testUnElementoEsCapicua() {
    assertTrue(Palindrome.capicua(new char[]{'x'}));
}

```

Por que estos cinco tests.

- `testCapicua`: caso positivo tipico (capicua impar de varios caracteres).
- `testNoCapicuaPar`: minimo no capicua de longitud par.
- `testNoCapicuaConExtremosIguales`: clave para evitar que la implementacion solo compare el primer y ultimo caracter. Si un mutante se queda con la primera comparacion, responderia `true` y este test lo detecta.
- `testArregloVacioEsCapicua` y `testUnElementoEsCapicua`: cubren los dos bordes ($l = 0$ y $l = 1$) que naturalmente devuelven `true` sin entrar al bucle.

1.3. Reporte de PIT

Metrica	Valor
Mutantes generados	9
Mutantes muertos	8
Mutantes sobrevivientes	1
<i>Mutation Score</i> bruto	8/9 = 89 %
<i>Mutation Score</i> sobre no equivalentes	8/8 = 100 %

Antes de completar la suite el score base (template) era $5/9 = 56\%$. Cada test que se sumo elimino mutantes especificos asociados al borde del bucle, a la negacion de la condicion del `if` y al manejo de arreglos cortos.

1.4. Respuestas

(a) ¿Cuantos mutantes genero la herramienta? **9 mutantes** sobre el metodo `capicua`.

(b) ¿Cuantos tests se necesitaron? **5 tests**, suficientes para matar todos los mutantes *no equivalentes*.

(c) ¿Que puntaje se obtuvo antes de analizar equivalentes? **89 %** (8/9). El unico mutante restante es el que se justifica como equivalente.

(d) **Mutantes equivalentes.** Hay **1 mutante equivalente**, asociado a la guarda del `while`:

<pre>Original: while (index < (l - 1) && res) Mutado: while (index <= (l - 1) && res)</pre>

Se analiza por casos sobre la longitud l :

1. $l = 0$: ambos guardas son falsas ($0 < -1$ y $0 \leq -1$). No se entra al ciclo. Ambas versiones retornan `true`.
2. $l = 1$: original no entra ($0 < 0$ es falso). Mutado entra una vez ($0 \leq 0$), compara `list[0]` con `list[(1-0)-1] = list[0]`, que es la misma posicion: la igualdad es trivial y `res` no cambia. Termina con `res = true` igual que el original.
3. $l \geq 2$: el bucle original ejecuta para $index = 0, 1, \dots, l-2$. El mutante ademas ejecuta una iteracion extra con $index = l-1$, donde se compara `list[l-1]` con `list[(1 - (1-1)) - 1] = list[0]`. Pero ese mismo par $(list[0], list[l-1])$ ya se comparo en la primera iteracion del bucle original. La comparacion adicional es por definicion

redundante: si los extremos eran iguales, lo siguen siendo; si eran distintos, `res` ya valdria `false` y la segunda parte del `&&` corta el ciclo.

Por lo tanto el mutante *no es distinguishable* por ningun input y no debe contarse contra el score: tras descontarlo, el resultado efectivo es $8/8 = 100\%$.

2. Ejercicio 2: `TriTyp.triang`

2.1. Codigo bajo prueba

El metodo `triang(s1,s2,s3)` clasifica un triangulo: 1 escaleno, 2 isosceles, 3 equilatero, 4 no triangulo (lados no positivos o desigualdad triangular incumplida). Su flujo encadena varios `if` sobre igualdades de lados, sumas y comparaciones con `triOut`.

2.2. Configuracion necesaria de PIT

El `pom.xml` apunta por defecto a `Palindrome`. Para este ejercicio se le indica explicitamente a PIT cual es la clase y la suite objetivo:

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -Dmaven.repo.local
=.m2 \
-Djacoco.skip=true \
-DtargetClasses=assignment6_exercises.TriTyp \
-DtargetTests=assignment6_exercises.TriTypTest \
test org.pitest:pitest-maven:mutationCoverage
```

2.3. Suite construida

Para llegar a *Mutation Score* completo hubo que pensar la suite por familias semanticas de casos. Se proponen **15 tests**, agrupados asi:

- **Equilatero valido:** `triang(3,3,3) = 3`.
- **Escaleno valido:** `triang(3,4,5) = 1`.
- **No triangulo (escaleno) por desigualdad triangular**, en las tres permutaciones donde la suma de dos lados iguala o queda por debajo del tercero: `triang(1,2,3) = 4`, `triang(5,2,3) = 4`, `triang(2,5,3) = 4`.
- **No triangulo (escaleno) por desigualdad estricta:** `triang(2,3,10) = 4`.
- **Isosceles valido en sus tres variantes:** `triang(2,2,3) = 2`, `triang(2,3,2) = 2`, `triang(3,2,2) = 2`.

- **Isosceles que no es triangulo** (par igual pero el tercero rompe la desigualdad):
`triang(1,1,2) = 4`, `triang(1,2,1) = 4`, `triang(2,1,1) = 4`.
- **Lado no positivo** en cada una de las tres posiciones: `triang(0,2,2) = 4`, `triang(2,0,2) = 4`, `triang(2,2,0) = 4`.

Cada familia ejercita una decision distinta del metodo (igualdades, sumas, `triOut` en $\{0, 1, 2, 3, > 3\}$, validez de lados), garantizando que cada operador de mutacion encuentre algun caso que lo detecte.

2.4. Reporte de PIT

Metrica	Valor
Mutantes generados	39
Mutantes muertos	39
Mutantes sobrevivientes	0
<i>Mutation Score</i>	$39/39 = 100\%$

Progreso: partiendo de un solo test (`triang(3,3,3)`) el score base era $9/39 \approx 23\%$. La adiccion de los 14 casos restantes lo llevo al 100%.

2.5. Respuestas

(a) ¿Cuantos mutantes? **39 mutantes.**

(b) ¿Cuantos tests? **15 tests** en `TriTypTest`.

(c) ¿Score antes de analizar equivalentes? **100 % (39/39).**

(d) **Mutantes equivalentes. 0 mutantes equivalentes observados.**

La justificacion es directa: la suite mato todos los mutantes generados, por lo que no quedaron sobrevivientes a clasificar como equivalentes. Esto no significa que el operador de mutacion sea incapaz de producir equivalentes para este metodo en general; simplemente que, en esta corrida, ninguno aparecio como sobreviviente. Si hubiera quedado alguno vivo, habria que analizar caso por caso si admite o no input distinguidor.

3. Ejercicio 3: Fuzzing

3.1. Estructura del paquete fuzzing

El paquete provee:

- **Fuzzer**: interfaz con un unico metodo `String fuzz()`.
- **Mutator**: clase utilitaria con operadores de mutacion sobre strings.
- **MutationFuzzer**: fuzzer basado en mutaciones a partir de un conjunto de *semillas* validas.
- **RandomFuzzer**: fuzzer puramente aleatorio.
- **HttpProgramRunner**: punto de comparacion — mide cuantos strings generados por cada fuzzer son URLs validas.
- **LinuxCommandTest**: test parametrizado que ejecuta `bc` sobre 100 entradas mutadas y chequea ausencia de crashes.

3.2. (a) Mutator

Tres operadores de mutacion sobre cadenas, todos protegidos contra entradas `null` y manejando explicitamente la cadena vacia donde corresponde:

1. `deleteRandomCharacter(s)` elige una posicion aleatoria $pos \in [0, |s|)$ y devuelve `s` sin ese caracter. Si $|s| = 0$, devuelve `s` inalterada.
2. `insertRandomCharacter(s)` elige $pos \in [0, |s|]$ (permite insertar al final) y un `char` aleatorio en el rango ASCII imprimible $[32, 126]$, y lo inserta.
3. `flipRandomCharacter(s)` elige $pos \in [0, |s|)$ y aplica XOR contra un solo bit pseudo-aleatorio de los siete bits bajos del caracter. Esto produce un caracter distinto en la misma posicion. Tambien protege la cadena vacia.

El metodo publico `mutate(s)` elige uniformemente entre los tres operadores y los delega.

Por que estos tres operadores. Forman una base minima que cubre los tres modos de cambio sintactico sobre un string: *eliminar* un simbolo, *agregar* uno, y *modificar* uno existente. Permiten alcanzar progresivamente toda cadena finita en pocos pasos sucesivos (encadenando `mutate` sobre el resultado anterior).

3.3. (b) MutationFuzzer

```
public String fuzz() {
    String candidate = population.get(random.nextInt(population.
        size()));
    int mutations = min_mutations;
    if (max_mutations > min_mutations) {
```

```

        mutations += random.nextInt((max_mutations -
            min_mutations) + 1);
    }
    for (int i = 0; i < mutations; i++) {
        candidate = Mutator.mutate(candidate);
    }
    return candidate;
}

```

Funcionamiento.

1. Elige una semilla al azar de la poblacion (cadenas inicialmente validas).
2. Determina cuantas mutaciones aplicar en este `fuzz()`, entre `min_mutations` y `max_mutations` (inclusivo).
3. Aplica las mutaciones *en cadena*: cada paso opera sobre el resultado anterior.

Correccion vs. el template original. El constructor del template tenia un bug por el cual `this.max_mutations` no se asignaba correctamente; el codigo final asigna los tres parametros explicitamente. Tambien se agregaron precondiciones (`population` no vacia, rango de mutaciones consistente).

3.4. (c) RandomFuzzer

```

public String fuzz() {
    int stringLength = random.nextInt(maxLength + 1);
    StringBuilder result = new StringBuilder(stringLength);
    for (int i = 0; i < stringLength; i++) {
        char c = (char) (charStart + random.nextInt(charRange));
        result.append(c);
    }
    return result.toString();
}

```

Funcionamiento.

1. Elige una longitud aleatoria $L \in [0, \text{maxLength}]$.
2. Construye una cadena de longitud L donde cada caracter es aleatorio en $[\text{charStart}, \text{charStart} + \text{charRange})$.

Es el contrapunto del `MutationFuzzer`: no parte de ninguna entrada valida y por lo tanto rara vez produce strings que “parezcan” validos.

3.5. (d) LinuxCommandTest

El test parametrizado ejecuta bc (calculadora Linux) sobre 100 cadenas generadas por el fuzzer de mutacion (que parte de la semilla "2 + 2").

```
@ParameterizedTest
@MethodSource("stringProvider")
public void bcCommandTest(String data) throws IOException,
    InterruptedException {
    String interactiveData = data + "\nquit";
    writer.write(interactiveData); writer.close();

    ProcessBuilder pb = new ProcessBuilder(program, FILE);
    Process process = pb.start();

    // ... captura stdout, stderr ...

    int exitCode = process.waitFor();

    assertEquals(134, exitCode); // abort
    assertEquals(139, exitCode); // segfault
    String stderrLower = stderr.toString().toLowerCase();
    assertFalse(stderrLower.contains("segmentation fault"));
    assertFalse(stderrLower.contains("core dumped"));
    assertFalse(stderrLower.contains("illegal instruction"));
}
```

Diseño del oráculo.

- No se exige `exitCode == 0`. Es normal que bc rechace entradas malformadas con error sintactico; eso *no es un crash*.
- Si se rechazan señales típicos de fallo grave: exit code 134 (SIGABRT), 139 (SIGSEGV), o mensajes de `stderr` que mencionen segmentation fault, core dumped o illegal instruction.

La consigna prohíbe expresamente cambiar bc por otro programa, ya que el fuzzer puede generar entradas peligrosas si se las pasara a un programa que modifica el sistema. bc es seguro: solo lee expresiones aritmeticas.

3.6. Ejecucion

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -Dmaven.repo.local
=.m2 \
```

```
-Djacoco.skip=true test
```

Resultado: BUILD SUCCESS. `LinuxCommandTest` pasa los 100 casos parametrizados y `TriTypTest` pasa sus 15 tests.

Conclusiones

- **Testing de mutacion** es una herramienta poderosa para evaluar la *capacidad de deteccion* de una suite: no alcanza con tener cobertura de lineas o de ramas. PIT inyecta variaciones sintacticas tipicas (cambiar operadores, eliminar asignaciones, mover constantes, negar condicionales), y un test debe ser capaz de distinguirlas. En el caso de `Palindrome`, una suite de 5 tests bien pensados llega a 89 % y deja un unico mutante *equivalente* identificable analiticamente.
- En `TriTyp` el score alcanzo el 100 %, pero esto fue resultado de una suite de 15 tests *enfocada* en las decisiones del codigo. Una cobertura ingenua (e.g. todos equilateros) deja la enorme mayoria de los mutantes vivos.
- **Fuzzing** es una tecnica complementaria: en lugar de pedir tests humanos, genera miles de entradas. `MutationFuzzer` y `RandomFuzzer` muestran el tradeoff entre *validez* y *exploracion*: las mutaciones de semilla preservan estructura (mas inputs validos), la generacion aleatoria explora mas pero acierta menos. La eleccion del oraculo (que considera un “fallo”) es tan importante como la generacion misma.